

## Experiment No. 3 : Cryptocurrency & P2P Blockchain Mining

Date: September 2, 2026

### 1 Aim

To create a cryptocurrency (Hadcoin) using Python, deploy it as a peer-to-peer network of three independent blockchain nodes, and perform mining, transaction handling, and chain-consensus across the network.

### 2 Theory

Lab 2 built a single blockchain running in one process. This experiment turns that into something closer to a real cryptocurrency network by running the *same* blockchain logic independently on three nodes (ports 5001, 5002, 5003) and letting them agree with each other over HTTP instead of sharing memory.

- **Independent ledgers:** Each node runs its own `Blockchain` instance with its own chain, its own pending transaction pool, and its own `node_address` (a random UUID that identifies it as the miner when it earns a reward). Nothing is shared by default – that is exactly the problem a P2P network has to solve.
- **Peer discovery:** `/connect_node` lets a node learn about other nodes on the network. Internally, `add_node()` uses `urlparse` to reduce a URL like `http://127.0.0.1:5002` to just its network location (`127.0.0.1:5002`) and stores it in a `set`, so the same peer is never registered twice.
- **Transactions and mining rewards:** `/add_transaction` queues a transaction on whichever node receives it. When that node mines a block, `mine_block()` first appends an automatic reward transaction (1 Hadcoin, paid to the miner's address) before sealing *all* pending transactions into the new block – so a miner is always compensated for the Proof-of-Work it performs.
- **Consensus – the longest chain rule:** Because each node mines independently, two nodes can end up with chains of different lengths. `replace_chain()` asks every known peer for its chain via `/get_chain`, and if a peer's chain is both *longer* and *valid* (checked with the same `is_chain_valid()` logic from Lab 2), the local node adopts it. This is what keeps every node's copy of the ledger converging on the same history without any central coordinator.

Compared to Lab 2, this file already keeps `create_block()` clearing `self.transactions` back to an empty list right after sealing them into a block (line 33 below), so a settled transaction can never be mined into a second block by mistake.

### 3 Program

The three node scripts (`hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`) are identical apart from the port they bind to and the name used for the mining reward's recipient. The listing below is `hadcoin_node_5001.py`; 5002 and 5003 differ only in `app.run(..., port = 5002/5003)` and the reward receiver string in `mine_block()`.

```
1 # Module 2 - Create a Cryptocurrency
2
3 # To be installed:
4 # Flask==0.12.2: pip install Flask==0.12.2
5 # Postman HTTP Client: https://www.getpostman.com/
6 # requests==2.18.4: pip install requests==2.18.4
7
```

```
8 # Importing the libraries
9 import datetime
10 import hashlib
11 import json
12 from flask import Flask, jsonify, request
13 import requests
14 from uuid import uuid4 #
15     Generate a unique id that is in hex
16 from urllib.parse import urlparse #
17     To parse url of the nodes
18
19 # Part 1 - Building a Blockchain
20
21 class Blockchain:
22
23     def __init__(self):
24         self.chain = []
25         self.transactions = [] #
26         Adding transactions before they are added to a block
27         self.create_block(proof = 1, previous_hash = '0')
28         self.nodes = set() #
29         Set is used as there is no order to be maintained as the nodes can be
30         from all around the globe
31
32     def create_block(self, proof, previous_hash):
33         block = {'index': len(self.chain) + 1,
34                 'timestamp': str(datetime.datetime.now()),
35                 'proof': proof,
36                 'previous_hash': previous_hash,
37                 'transactions': self.transactions} #
38         Adding transactions to make the blockchain a cryptocurrency
39         self.transactions = [] #
40         The list of transaction should become empty after they are added to a
41         block
42         self.chain.append(block)
43         return block
44
45     def get_previous_block(self):
46         return self.chain[-1]
47
48     def proof_of_work(self, previous_proof):
49         new_proof = 1
50         check_proof = False
51         while check_proof is False:
52             hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).
53                                               encode()).hexdigest()
54             if hash_operation[:4] == '0000':
55                 check_proof = True
56             else:
57                 new_proof += 1
58         return new_proof
59
60     def hash(self, block):
61         encoded_block = json.dumps(block, sort_keys = True).encode()
62         return hashlib.sha256(encoded_block).hexdigest()
63
64     def is_chain_valid(self, chain):
65         previous_block = chain[0]
66         block_index = 1
67         while block_index < len(chain):
68             block = chain[block_index]
69             if block['previous_hash'] != self.hash(previous_block):
70                 return False
71             previous_block = block
72             block_index += 1
```

```

62         previous_proof = previous_block['proof']
63         proof = block['proof']
64         hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).
65             encode()).hexdigest()
66         if hash_operation[:4] != '0000':
67             return False
68         previous_block = block
69         block_index += 1
70     return True
71
72     # This method will add the transaction to the list of transactions
73     def add_transaction(self, sender, receiver, amount):
74         self.transactions.append({'sender': sender,
75             'receiver': receiver,
76             'amount': amount})
77         previous_block = self.get_previous_block()
78         return previous_block['index'] + 1 #
79         It will return the block index to which the transaction should be added
80
81     # This function will add the node containing an address to the set of nodes
82     created in init function
83     def add_node(self, address):
84         parsed_url = urlparse(address) #
85         urlparse will parse the url from the address
86         self.nodes.add(parsed_url.netloc) #
87         Add is used and not append as it's a set. Netloc will only return
88         '127.0.0.1:5000'
89
90     # Consensus Protocol. This function will replace all the shorter chain with the
91     longer chain in all the nodes on the network
92     def replace_chain(self):
93         network = self.nodes #
94         network variable is the set of nodes all around the globe
95         longest_chain = None #
96         It will hold the longest chain when we scan the network
97         max_length = len(self.chain) #
98         This will hold the length of the chain held by the node that runs this
99         function
100         for node in network:
101             response = requests.get(f'http://{node}/get_chain') #
102             Use get chain method already created to get the length of the
103             chain
104             if response.status_code == 200:
105                 length = response.json()['length'] #
106                 Extract the length of the chain from get_chain function
107                 chain = response.json()['chain']
108                 if length > max_length and self.is_chain_valid(chain): #
109                     We check if the length is bigger and if the chain is valid then
110                     max_length = length #
111                     We update the max length
112                     longest_chain = chain #
113                     We update the longest chain
114             if longest_chain: #
115                 If longest_chain is not none that means it was replaced
116                 self.chain = longest_chain #
117                 Replace the chain of the current node with the longest chain
118                 return True
119         return False #
120         Return false if current chain is the longest one
121
122     # Part 2 - Mining our Blockchain
123
124     # Creating a Web App

```

```

105 app = Flask(__name__)
106
107 # Creating an address for the node on Port 5000. We will create some other nodes as
    well on different ports
108 node_address = str(uuid4()).replace('-', '') #
109
110 # Creating a Blockchain
111 blockchain = Blockchain()
112
113 # Mining a new block
114 @app.route('/mine_block', methods = ['GET'])
115 def mine_block():
116     previous_block = blockchain.get_previous_block()
117     previous_proof = previous_block['proof']
118     proof = blockchain.proof_of_work(previous_proof)
119     previous_hash = blockchain.hash(previous_block)
120     blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount
        = 1) # Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin
        to Abcde for mining the block
121     block = blockchain.create_block(proof, previous_hash)
122     response = {'message': 'Congratulations, you just mined a block!',
123                 'index': block['index'],
124                 'timestamp': block['timestamp'],
125                 'proof': block['proof'],
126                 'previous_hash': block['previous_hash'],
127                 'transactions': block['transactions']}
128     return jsonify(response), 200
129
130 # Getting the full Blockchain
131 @app.route('/get_chain', methods = ['GET'])
132 def get_chain():
133     response = {'chain': blockchain.chain,
134                 'length': len(blockchain.chain)}
135     return jsonify(response), 200
136
137 # Checking if the Blockchain is valid
138 @app.route('/is_valid', methods = ['GET'])
139 def is_valid():
140     is_valid = blockchain.is_chain_valid(blockchain.chain)
141     if is_valid:
142         response = {'message': 'All good. The Blockchain is valid.'}
143     else:
144         response = {'message': 'Houston, we have a problem. The Blockchain is not
            valid.'}
145     return jsonify(response), 200
146
147 # Adding a new transaction to the Blockchain
148 @app.route('/add_transaction', methods = ['POST']) #
149     Post method as we have to pass something to get something in return
150 def add_transaction():
151     json = request.get_json() #
152     This will get the json file from postman. In Postman we will create a json
        file in which we will pass the values for the keys in the json file
153     transaction_keys = ['sender', 'receiver', 'amount']
154     if not all(key in json for key in transaction_keys): #
155         Checking if all keys are available in json
156         return 'Some elements of the transaction are missing', 400
157     index = blockchain.add_transaction(json['sender'], json['receiver'], json['
        amount'])
158     response = {'message': f'This transaction will be added to Block {index}'}
159     return jsonify(response), 201 #
160     Code 201 for creation

```

```

158 # Part 3 - Decentralizing our Blockchain
159
160 # Connecting new nodes
161 @app.route('/connect_node', methods = ['POST']) #
162     POST request to register the new nodes from the json file
163 def connect_node():
164     json = request.get_json()
165     nodes = json.get('nodes') #
166     Get the nodes from json file
167     if nodes is None:
168         return "No node", 400
169     for node in nodes:
170         blockchain.add_node(node)
171     response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain
172                 now contains the following nodes:',
173                 'total_nodes': list(blockchain.nodes)}
174     return jsonify(response), 201
175
176 # Replacing the chain by the longest chain if needed
177 @app.route('/replace_chain', methods = ['GET'])
178 def replace_chain():
179     is_chain_replaced = blockchain.replace_chain()
180     if is_chain_replaced:
181         response = {'message': 'The nodes had different chains so the chain was
182                     replaced by the longest one.',
183                     'new_chain': blockchain.chain}
184     else:
185         response = {'message': 'All good. The chain is the largest one.',
186                     'actual_chain': blockchain.chain}
187     return jsonify(response), 200
188
189 # Running the app
190 app.run(host = '0.0.0.0', port = 5001)

```

Listing 1: hadcoin\_node\_5001.py – Hadcoin blockchain node, P2P & consensus logic

## 4 Output

All three nodes were started locally in separate terminals, connected to each other as peers via Postman, and then exercised end-to-end: transactions were added and mined on one node only, so that node's chain would pull ahead, and /replace\_chain was used to bring the other two nodes back in sync.

### 1. Setting up the environment

```

jay@lab-machine: ~/blockchain-lab-3

(venv) jay@lab-machine:~/blockchain-lab-3$ pip install requests==2.18.4
Collecting requests==2.18.4
  Downloading requests-2.18.4-py2.py3-none-any.whl (88 kB)
Collecting idna<2.7,>=2.5
Collecting urllib3<1.23,>=1.21.1
Collecting certifi>=2017.4.17
Collecting chardet<3.1.0,>=3.0.2
Installing collected packages: idna, urllib3, certifi, chardet, requests
Successfully installed certifi-2024.2.2 chardet-3.0.4 idna-2.6 requests-2.18.4 urllib3-1.22

(venv) jay@lab-machine:~/blockchain-lab-3$ ls Lab_3_Create_a_Cryptocurrency
hadcoin.py hadcoin_node_5001.py hadcoin_node_5002.py hadcoin_node_5003.py nodes.json transaction.json

```

Installing requests and confirming the Lab 3 files

## 2. Starting the three nodes

Each node script was run in its own terminal, binding to ports 5001, 5002 and 5003 respectively.

```
Terminal 1 -- jay@lab-machine

(venv) jay@lab-machine:~/blockchain-lab-3$ python3 hadcoin_node_5001.py
* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
```

*Node 1 – hadcoin\_node\_5001.py on port 5001*

```
Terminal 2 -- jay@lab-machine

(venv) jay@lab-machine:~/blockchain-lab-3$ python3 hadcoin_node_5002.py
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

*Node 2 – hadcoin\_node\_5002.py on port 5002*

```
Terminal 3 -- jay@lab-machine

(venv) jay@lab-machine:~/blockchain-lab-3$ python3 hadcoin_node_5003.py
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
Press CTRL+C to quit
```

*Node 3 – hadcoin\_node\_5003.py on port 5003*

## 3. Connecting the nodes into a network

Each node was told about its two peers via POST /connect\_node.

POST

127.0.0.1:5001/connect\_node

Send

REQUEST BODY (JSON)

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5003"
  ]
}
```

Response Status: 201 Created

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5003",
    "127.0.0.1:5002"
  ]
}
```

*POST /connect\_node on 5001 – registers 5002 and 5003 as peers*

|                                                                                                                                                                                            |                             |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|------|
| POST                                                                                                                                                                                       | 127.0.0.1:5002/connect_node | Send |
| <b>REQUEST BODY (JSON)</b>                                                                                                                                                                 |                             |      |
| <pre>{   "nodes": [     "http://127.0.0.1:5001",     "http://127.0.0.1:5003"   ] }</pre>                                                                                                   |                             |      |
| <b>Response Status: 201 Created</b>                                                                                                                                                        |                             |      |
| <pre>{   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",   "total_nodes": [     "127.0.0.1:5003",     "127.0.0.1:5001"   ] }</pre> |                             |      |

*POST /connect\_node on 5002 – registers 5001 and 5003 as peers*

|                                                                                                                                                                                            |                             |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|------|
| POST                                                                                                                                                                                       | 127.0.0.1:5003/connect_node | Send |
| <b>REQUEST BODY (JSON)</b>                                                                                                                                                                 |                             |      |
| <pre>{   "nodes": [     "http://127.0.0.1:5001",     "http://127.0.0.1:5002"   ] }</pre>                                                                                                   |                             |      |
| <b>Response Status: 201 Created</b>                                                                                                                                                        |                             |      |
| <pre>{   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",   "total_nodes": [     "127.0.0.1:5002",     "127.0.0.1:5001"   ] }</pre> |                             |      |

*POST /connect\_node on 5003 – registers 5001 and 5002 as peers*

#### 4. Queuing transactions on Node 5001

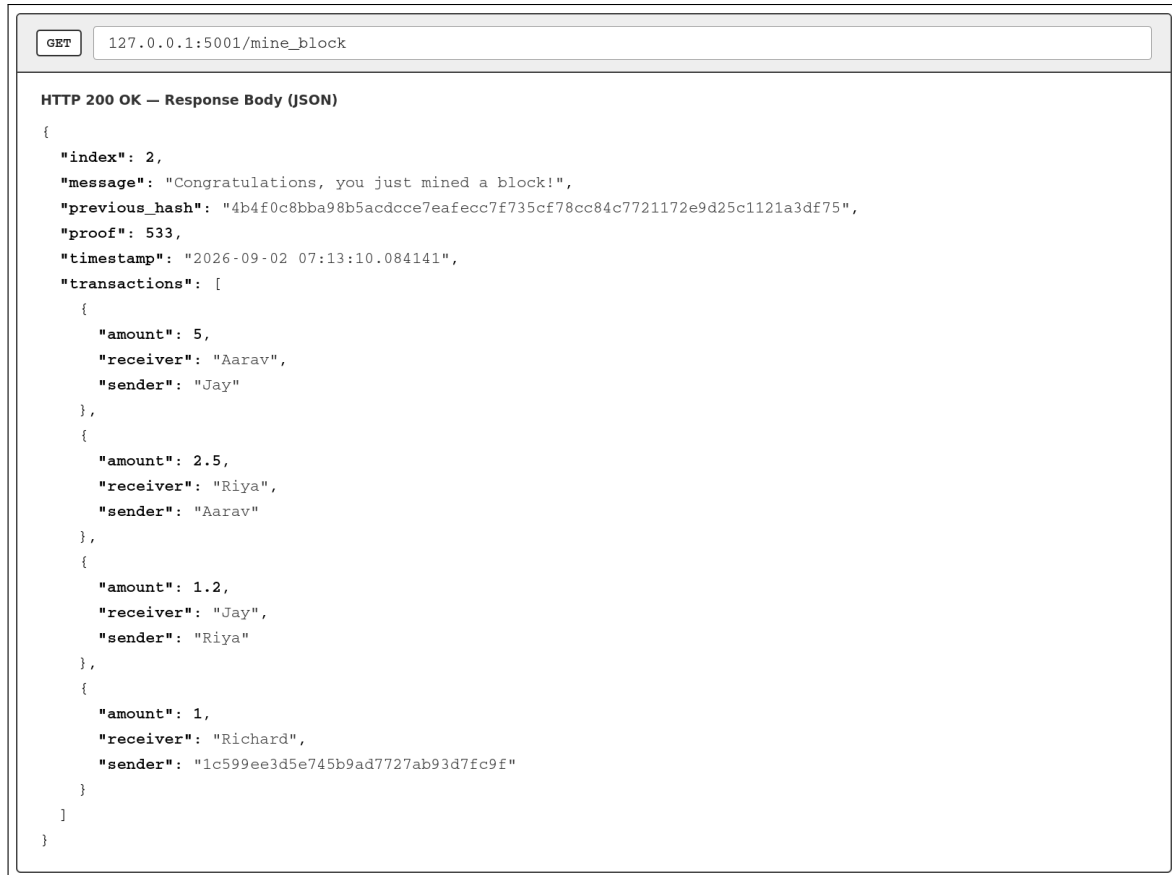
Three transactions were queued on node 5001 only (Jay→Aarav, Aarav→Riya, Riya→Jay), one POST /add\_transaction call each. The first is shown below; the other two follow the identical request/response pattern, targeting Block 2.

|                                                                         |                                |      |
|-------------------------------------------------------------------------|--------------------------------|------|
| POST                                                                    | 127.0.0.1:5001/add_transaction | Send |
| <b>REQUEST BODY (JSON)</b>                                              |                                |      |
| <pre>{   "sender": "Jay",   "receiver": "Aarav",   "amount": 5 }</pre>  |                                |      |
| <b>Response Status: 201 Created</b>                                     |                                |      |
| <pre>{   "message": "This transaction will be added to Block 2" }</pre> |                                |      |

*POST /add\_transaction on 5001 – queuing 5 Hadcoins from Jay to Aarav*

## 5. Mining on Node 5001 – creating a length mismatch

With three transactions pending, GET /mine\_block was called only on node 5001. This settles all three queued transactions plus the automatic 1-Hadcoin mining reward into Block 2, so 5001's chain length becomes **2** while 5002 and 5003 remain at length **1**.



```
GET 127.0.0.1:5001/mine_block

HTTP 200 OK — Response Body (JSON)

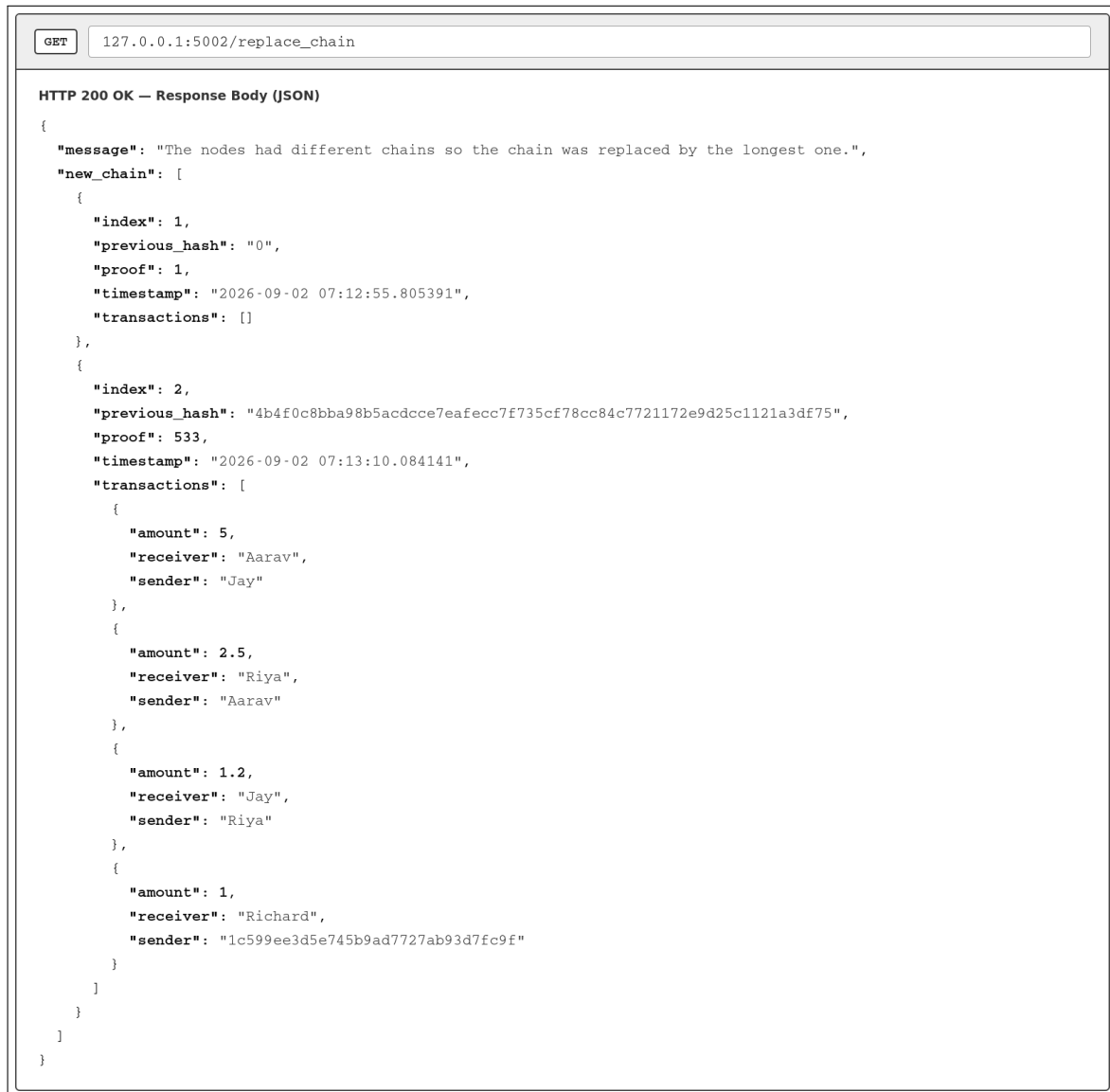
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "4b4f0c8bba98b5acdce7eafecc7f735cf78cc84c7721172e9d25c1121a3df75",
  "proof": 533,
  "timestamp": "2026-09-02 07:13:10.084141",
  "transactions": [
    {
      "amount": 5,
      "receiver": "Aarav",
      "sender": "Jay"
    },
    {
      "amount": 2.5,
      "receiver": "Riya",
      "sender": "Aarav"
    },
    {
      "amount": 1.2,
      "receiver": "Jay",
      "sender": "Riya"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "1c599ee3d5e745b9ad7727ab93d7fc9f"
    }
  ]
}
```

*GET /mine\_block on 5001 – Block 2 mined with 3 transactions + reward*

## 6. Restoring consensus with the longest-chain rule

Calling GET /replace\_chain on the two lagging nodes makes each of them fetch every peer's chain, find that 5001's chain is longer and valid, and adopt it.



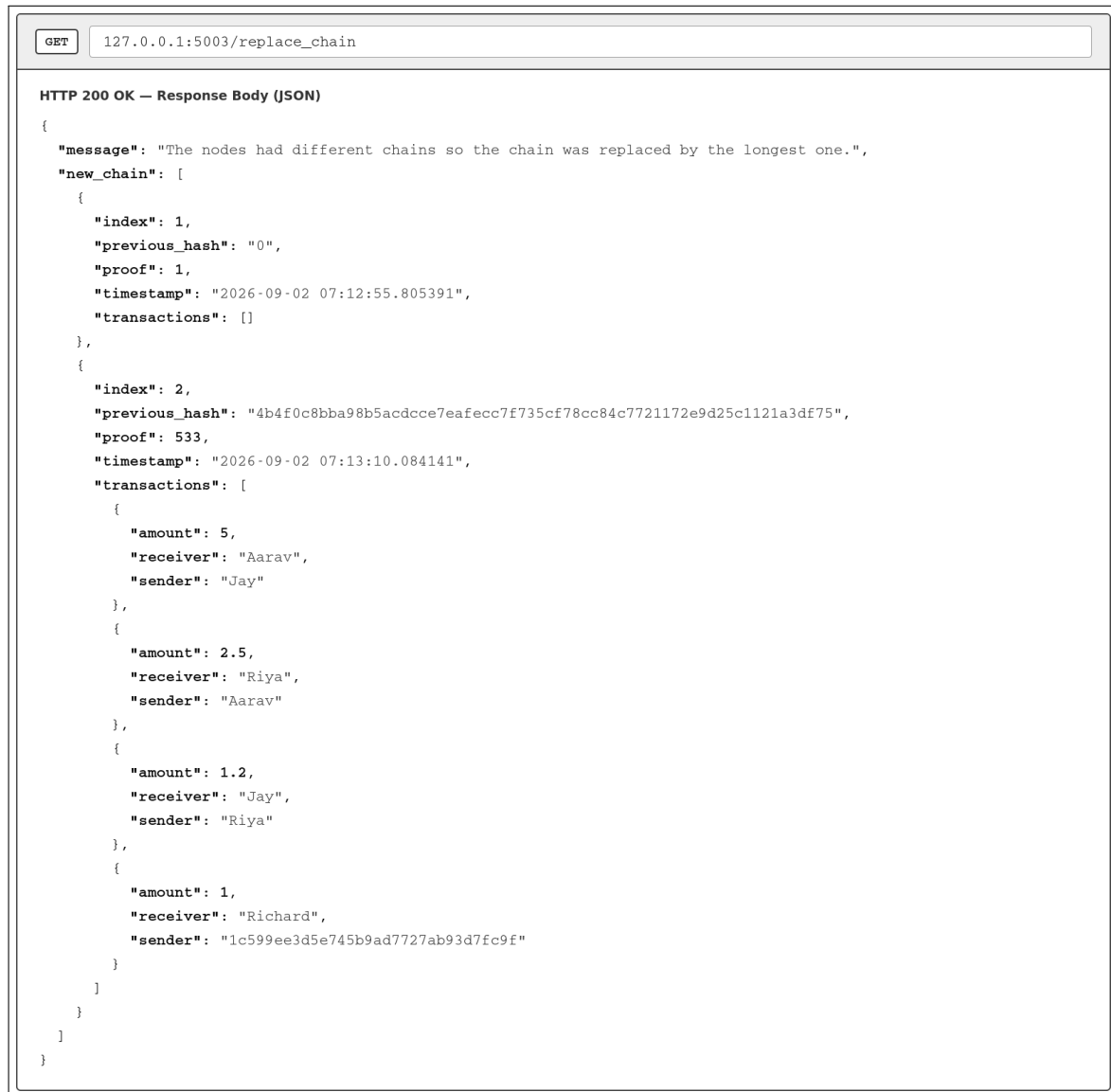


```
GET 127.0.0.1:5002/replace_chain

HTTP 200 OK — Response Body (JSON)

{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 07:12:55.805391",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "4b4f0c8bba98b5acdce7eafec7f735cf78cc84c7721172e9d25c1121a3df75",
      "proof": 533,
      "timestamp": "2026-09-02 07:13:10.084141",
      "transactions": [
        {
          "amount": 5,
          "receiver": "Aarav",
          "sender": "Jay"
        },
        {
          "amount": 2.5,
          "receiver": "Riya",
          "sender": "Aarav"
        },
        {
          "amount": 1.2,
          "receiver": "Jay",
          "sender": "Riya"
        },
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "1c599ee3d5e745b9ad7727ab93d7fc9f"
        }
      ]
    }
  ]
}
```

*GET /replace\_chain on 5002 – adopts 5001's longer chain*



```
GET 127.0.0.1:5003/replace_chain

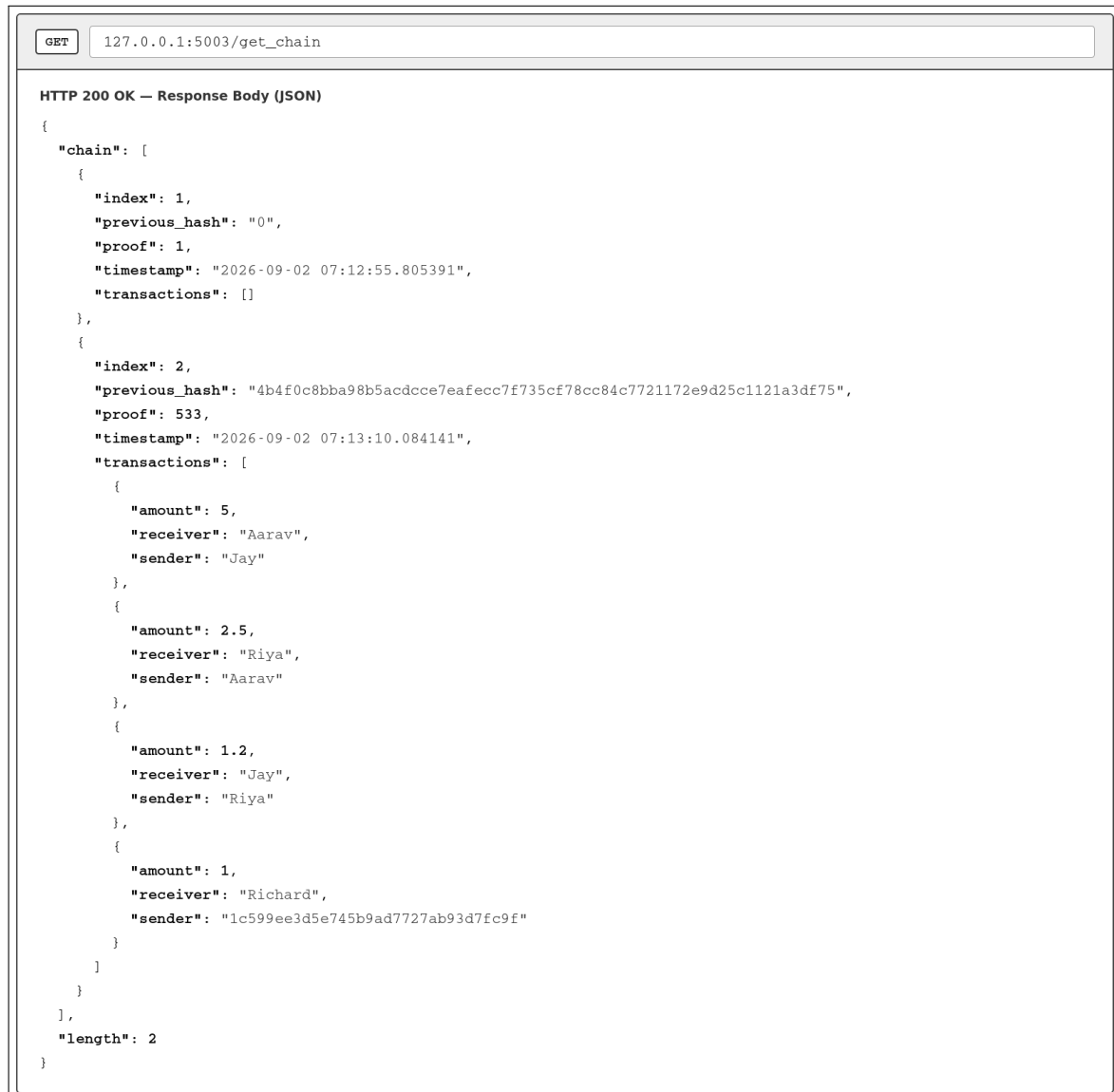
HTTP 200 OK — Response Body (JSON)

{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 07:12:55.805391",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "4b4f0c8bba98b5acdce7eafecc7f735cf78cc84c7721172e9d25c1121a3df75",
      "proof": 533,
      "timestamp": "2026-09-02 07:13:10.084141",
      "transactions": [
        {
          "amount": 5,
          "receiver": "Aarav",
          "sender": "Jay"
        },
        {
          "amount": 2.5,
          "receiver": "Riya",
          "sender": "Aarav"
        },
        {
          "amount": 1.2,
          "receiver": "Jay",
          "sender": "Riya"
        },
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "1c599ee3d5e745b9ad7727ab93d7fc9f"
        }
      ]
    }
  ]
}
```

*GET /replace\_chain on 5003 – adopts 5001's longer chain*

## 7. Confirming the network is synchronised

GET /get\_chain now returns the identical two-block chain (length 2) from every node in the network.



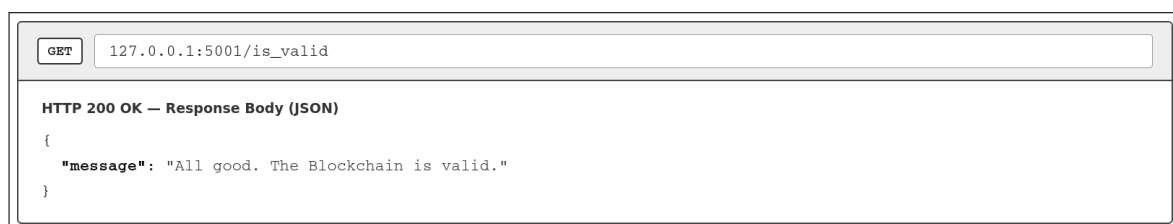
```
GET 127.0.0.1:5003/get_chain

HTTP 200 OK — Response Body (JSON)

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 07:12:55.805391",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "4b4f0c8bba98b5acdce7eafec7f735cf78cc84c7721172e9d25c1121a3df75",
      "proof": 533,
      "timestamp": "2026-09-02 07:13:10.084141",
      "transactions": [
        {
          "amount": 5,
          "receiver": "Aarav",
          "sender": "Jay"
        },
        {
          "amount": 2.5,
          "receiver": "Riya",
          "sender": "Aarav"
        },
        {
          "amount": 1.2,
          "receiver": "Jay",
          "sender": "Riya"
        },
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "1c599ee3d5e745b9ad7727ab93d7fc9f"
        }
      ]
    }
  ],
  "length": 2
}
```

*GET /get\_chain – all three nodes report the same synchronised chain*

## 8. Validating the chain



```
GET 127.0.0.1:5001/is_valid

HTTP 200 OK — Response Body (JSON)

{
  "message": "All good. The Blockchain is valid."
}
```

*GET /is\_valid on 5001 – integrity check passes*

## 5 Conclusion

A cryptocurrency (Hadcoin) was implemented in Python and deployed as three independent Flask nodes, each maintaining its own copy of the blockchain and communicating over HTTP rather than shared memory. Connecting the nodes via `/connect_node` and letting transactions accumulate and get mined on only one node made the divergence between chains easy to produce on purpose, which in turn made the value of the longest-chain consensus rule obvious: a single call to `/replace_chain`

on each lagging node was enough to pull the whole network back into agreement, without any node ever needing to trust another node directly – it only trusted whichever valid chain was demonstrably the longest. This mirrors, in miniature, exactly the problem real P2P cryptocurrency networks solve at a much larger scale: keeping thousands of independent nodes converged on one shared transaction history with no central authority in charge.